08. 에러 처리 및 보안 고려사항

에러 처리 전략

에러 타입 계층구조

이 구현체는 `anyhow` 크레이트를 사용하여 에러 체인과 컨텍스트를 제공합니다.

```
use anyhow::{anyhow, bail, Context, Result};


// 기본 에러 생성
return Err(anyhow!("Custom error message"));


// 조건부 에러 (early return)
if  condition {
bail!("Error condition met");
}


// 컨텍스트 추가
some_operation().context("Failed to perform operation")?;
```

주요 에러 카테고리

1. 파싱 에러

```rust
// Quote 파싱 실패
let quote = Quote::decode(&mut quote).context("Failed to decode quote")?;

// 인증서 파싱 실패
let cert = webpki::EndEntityCert::try_from(&cert_der)
.context("Failed to parse PCK certificate")?;

// JSON 파싱 실패
let tcb_info = serde_json::from_str::<TcbInfo>(&collateral.tcb_info)
.context("Failed to decode TcbInfo")?;
```

2. 암호학적 검증 에러

```rust
// 서명 검증 실패
peer_public_key
.verify(signed_data, signature)
.map_err(|_| anyhow!("Isv enclave report signature is invalid"))?;

// 인증서 체인 검증 실패
verify_certificate_chain(&leaf_cert, &intermediate_certs, now, &crls)
.context("Failed to verify certificate chain")?;

// 해시 불일치
```

```
if  qe_hash.as_ref() != &qe_report.report_data[0..32] {
bail!("QE report hash mismatch");
}
```

3. 데이터 무결성 에러

```
// FMSPC 불일치
if  fmspc != tcb_fmspc[..] {
bail!("Fmspc mismatch");
}

// 인증서 체인 길이 부족
if  qe_certification_certs.len() < 2 {
bail!("Certificate chain is too short in quote");
}

// 지원하지 않는 형식
if  certification_data.cert_type != PCK_CERT_CHAIN {
bail!("Unsupported DCAP PCK cert format");
}
```

4. 시간 관련 에러

```
// TCB 정보 만료
```

```
let next_update = chrono::DateTime::parse_from_rfc3339(&tcb_info.next_update)
.ok()
.context("Failed to parse next update")?;
if now_secs > next_update.timestamp() as u64 {
bail!("TCBInfo expired");
}
```

보안 고려사항

1. 타이밍 공격 방지

상수 시간 비교

```
// Ring 라이브러리의 상수 시간 구현 사용
use ring::signature::UnparsedPublicKey;

// 해시 비교 시 타이밍 공격 방지
if qe_hash.as_ref() != &qe_report.report_data[0..32] {
bail!("QE report hash mismatch");
}
```

조기 반환 방지

```
// 모든 검증 단계를 완료한 후 결과 반환
```

```
let mut validation_errors = Vec::new();


// 각 검증 단계에서 에러 수집
if !signature_valid {
validation_errors.push("Invalid signature");
}
if !cert_chain_valid {
validation_errors.push("Invalid certificate chain");
}


// 모든 검증 완료 후 에러 처리
if !validation_errors.is_empty() {
bail!("Validation failed: {:?}", validation_errors);
}
```

2. 메모리 안전성

버퍼 오버플로우 방지

```
// 안전한 슬라이싱
raw_quote
.get(..signed_quote_len)
.ok_or(anyhow!("Failed to get signed quote"))?;
```

```
// 배열 길이 검증
if data.len() != 16 {
bail!("CpuSvn length mismatch");
}
```

민감한 데이터 처리

```
// 개인키나 민감한 데이터는 즉시 제로화
use zeroize::Zeroize;

let mut sensitive_data = vec![0u8; 32];
// ... 사용 후
sensitive_data.zeroize();
```

3. 입력 검증

Quote 버전 검증

```
// 지원하는 Quote 버전만 허용
if ![3, 4, 5].contains(&quote.header.version) {
return Err(anyhow!("Unsupported DCAP quote version"));
}
```

TEE 타입 검증

```
// 알려진 TEE 타입만 허용
match header.tee_type {
TEE_TYPE_SGX | TEE_TYPE_TDX => { /* 허용 */ }
_ => return Err(scale::Error::from("Invalid TEE type")),
}
```

증명 키 타입 검증

```
// 지원하는 암호학적 알고리즘만 허용
if quote.header.attestation_key_type != ATTESTATION_KEY_TYPE_ECDSA256_WITH_P256
bail!("Unsupported DCAP attestation key type");
}
```

4. 인증서 검증 강화

CRL 검증 정책

```
let revocation = builder
.with_depth(webpki::RevocationCheckDepth::Chain) // 전체 체인 검사
.with_status_policy(webpki::UnknownStatusPolicy::Deny) // 알 수 없는 상태 거부
.with_expiration_policy(webpki::ExpirationPolicy::Enforce) // 만료 정책 강제
.build();
```

신뢰 앵커 고정

```
// 하드코딩된 Intel 루트 CA만 신뢰
pub static TRUSTED_ROOT_CA_DER: &[u8] = include_bytes!("TrustedRootCA.der");


// 신뢰 앵커 검증
leaf_cert.verify_for_usage(
sig_algs,
&[sgx_pck_root()], // 고정된 신뢰 앵커만 사용
intermediate_certs,
time,
webpki::KeyUsage::server_auth(),
Some(revocation),
None,
)
```

5. 네트워크 보안

HTTPS 강제 사용

```
// PCCS URL 검증
if !pccs_url.starts_with("https://") {
bail!("PCCS URL must use HTTPS");
}
```

타임아웃 설정

```
let builder = reqwest::Client::builder()
.timeout(Duration::from_secs(180)) // 3분 타임아웃
.build()?;
```

인증서 검증 (프로덕션)

```
// 개발 환경에서만 인증서 검증 비활성화
#[cfg(not(feature = "js"))]
let builder = builder
.danger_accept_invalid_certs(true) // 개발용만
.timeout(Duration::from_secs(180));
```

6. 속성 검증 강화

디버그 모드 거부

```
// SGX 디버그 모드 확인
fn validate_sgx(report: &EnclaveReport) -> Result<()> {
let is_debug = report.attributes[0] & 0x02 != 0;
if is_debug {
bail!("Debug mode is enabled");
}
Ok(())
}
```

```
// TDX 디버그 모드 확인
fn validate_td10(report: &TDReport10) -> Result<()> {
let td_attrs = TDAttributes::parse(report.td_attributes)?;
if td_attrs.tud != 0 {
bail!("Debug mode is enabled");
}
// ...
}
```

예약 비트 검증

```
// TDX 예약 비트 확인
if td_attrs.sec.reserved_lower != 0
|| td_attrs.sec.reserved_bit29
|| td_attrs.other.reserved != 0
{
bail!("Reserved bits in TD attributes are set");
}
```

7. 리플레이 공격 방지

타임스탬프 검증

```
// TCB 정보 만료 확인
if now_secs > next_update.timestamp() as u64 {
bail!("TCBInfo expired");
}
```

Nonce 사용 권장

```
// Report 데이터에 nonce 포함 권장
// 클라이언트가 report_data에 랜덤 nonce 포함해야 함
let nonce = &report.report_data[32..64]; // 사용자 정의 nonce
```

8. 사이드 채널 공격 방지

일정한 실행 시간

```
// 모든 검증 단계를 동일한 시간에 수행
let mut all_checks_passed = true;

// 각 검증을 독립적으로 수행
let signature_valid = verify_signature();
let cert_valid = verify_certificate();
let tcb_valid = verify_tcb();

all_checks_passed &= signature_valid;
```

```
all_checks_passed &= cert_valid;
all_checks_passed &= tcb_valid;

if !all_checks_passed {
bail!("Verification failed");
}
```

9. 로깅 및 감사

보안 이벤트 로깅

```
use log::{info, warn, error};

// 성공적인 검증 로깅
info!("Quote verification successful for FMSPC: {}", hex::encode(fmspc));

// 실패한 검증 로깅 (민감한 정보 제외)
warn!("Quote verification failed: {}", error_message);

// 보안 위반 로깅
error!("Security violation detected: debug mode enabled");
```

감사 추적

```
// 검증 과정의 주요 단계 기록
struct VerificationAudit {
quote_version: u16,
tee_type: u32,
fmspc: String,
tcb_status: String,
verification_time: u64,
}
```

10. 에러 정보 노출 제한

민감한 정보 마스킹

```
// 에러 메시지에서 민감한 정보 제거
.map_err(|_| anyhow!("Signature verification failed")) // 구체적인 암호학적 에러 숨

// 디버그 정보는 개발 환경에서만
#[cfg(debug_assertions)]
eprintln!("Debug info: {:?}", sensitive_data);
```

일관된 에러 응답

```
// 다양한 실패 원인에 대해 일관된 응답
match  verification_result {
```

```
Ok(report) => Ok(report),
Err(_) => Err(anyhow!("Quote verification failed")), // 구체적인 원인 숨김
}
```

11. 리소스 제한

메모리 사용량 제한

```
// 큰 데이터 구조 크기 제한
if cert_chain.len() > MAX_CERT_CHAIN_SIZE {
bail!("Certificate chain too large");
}
```

처리 시간 제한

```
use std::time::Instant;

let start_time = Instant::now();
// ... 검증 수행
if start_time.elapsed() > Duration::from_secs(30) {
bail!("Verification timeout");
}
```

이러한 보안 고려사항들을 통해 DCAP Quote 검증 과정에서 발생할 수 있는 다양한 보안 위협을 방지하고, 안전하고 신뢰할 수 있는 검증 결과를 제공할 수 있습니다.